PORTFOLIO

Isabel Esther Rodríguez Alvarez

Backend & AI Agent Engineer

Python · FastAPI · Microsoft Azure · RAG · Multi-agent · MCP

Software engineer with 5 years of experience in backend development and AI agents. I design APIs and cloud services on Azure (Functions, Service Bus, SignalR, App Services, AI Search, Cosmos DB) and build AI agents that run 24/7 in production: from multi-agent systems for healthcare to assistants that run real business processes. This portfolio presents four projects with their architecture, technical decisions and implementation work.

Remote · Cuba (US Eastern Time) · isabelestherr@gmail.com · LinkedIn · GitHub

5 years
of backend and AI experience

4
real-world case studies

64
custom MCP tools in a single agent

160k+
addresses and places indexed for search

1 **VitalIA** AI-powered healthcare platform

2 **Xuxa** AI real estate advisor · ED Bienes Raíces (Panama)

3 **Sage** Inventory and purchasing agent for restaurants (Panama)

4 **Backend Projects** Mobility platform: geospatial search and real-time messaging

Skills

Backend

FastAPI · Django · Spring · Node.js · REST APIs · Event-driven architecture · Clean Architecture / DDD

Microsoft Azure

Functions · Service Bus · SignalR · App Services · AI Search · Cosmos DB · Blob Storage · Key Vault · Container Registry · AI Foundry

AI Agents

Multi-agent · Hybrid RAG · MCP · Function calling · Agent memory · Azure AI Foundry Agents · Azure OpenAI

Data

PostgreSQL

MongoDB / Cosmos DB

Redis

Vector and geospatial search

DevOps

Docker

Azure DevOps (CI/CD)

Application Insights

OpenTelemetry

> Due to confidentiality agreements, no source code or client data is shown. Screenshots use test or anonymized data.

AI AGENTS + BACKEND

VitalIA

AI-powered healthcare platform

Multi-agent medical board, automated clinical reports and real-time medical chat.

Multi-agent | Azure AI Foundry | FastAPI | Azure Functions | SignalR | Service Bus | FHIR

3
microservices on Azure

~9
specialist agents on the medical board

8
report agents running in parallel

~40k
lines of code in the agent engine

My role

> Lead architect and developer of the platform’s three services: agent engine, orchestrator and real-time layer.

Context and problem

VitalIA is a healthcare platform that supports clinicians and patients. It needed to analyze complex clinical cases from several medical perspectives, generate periodic reports from standard clinical data (FHIR) and offer a conversational assistant. All with traceable answers, because in healthcare it is not enough for the AI to "sound right", and with real-time feedback while the AI works.

Solution

Multi-agent medical board

A coordinator agent analyzes the case and selects which specialists take part. Specialists propose hypotheses, critic agents for each specialty challenge them, the hypotheses are refined, and a consensus agent writes a clinical report with a calculated confidence level.

Subscription-based clinical reports

Eight agents generate vital-sign alerts, population trends, condition statistics and follow-ups in parallel, at the frequency and volume defined by the client's plan.

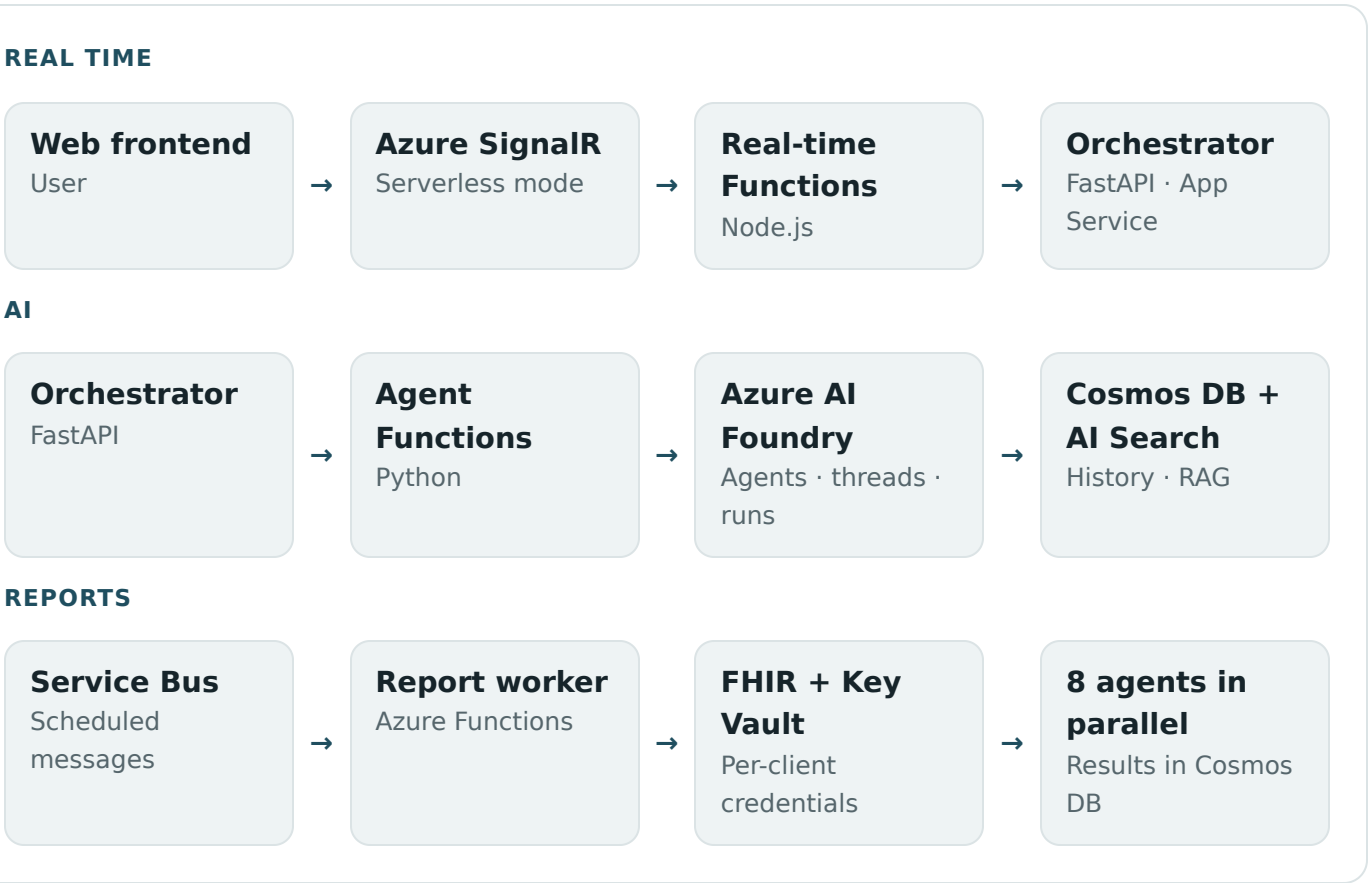
Medical chat with memory

Conversational assistant with persistent history, automatic titles and personalized question suggestions based on previous conversations.

Real time

Users see the analysis progress (phase, cycle and round) and the agent's answers as they are generated, through Azure SignalR.

Architecture



Overall architecture: three decoupled services communicating over HTTP, SignalR and Service Bus.

Multi-agent medical board flow

1
Clinical extraction
Text Analytics for Health identifies medical entities in the case.

2
Coordinator
Selects specialists, urgency level and red flags.

3
Round 0 · Hypotheses
Specialists propose diagnoses in parallel.

4
Round 1 · Critique
Specialty critic agents look for weaknesses and risks.

5
Round 2 · Refinement
Specialists adjust hypotheses and confidence.

6
Consensus
SOAP report with ranked hypotheses and calculated confidence.

Technologies

Agents and AI
Azure AI Foundry Agent Service, GPT-4o and GPT-4o-mini depending on the agent’s role, Azure OpenAI (embeddings), File Search with vector stores, Azure AI Language (Text Analytics for Health)
Backend
Python, FastAPI, Azure Functions (Python and Node.js), Pydantic, async httpx
Data
Cosmos DB (MongoDB API), Azure AI Search (vector index), Redis, Blob Storage
Messaging and real time
Azure SignalR Serverless, Azure Service Bus (topics and scheduled messages)
Security and integration
Key Vault, Managed Identity, FHIR R4/R5 standard

DevOps
Multi-stage Docker, Azure Pipelines, Container Registry, App Service, Application Insights
Architecture
Clean Architecture, DDD, CQRS and dependency injection

Implementation: challenges and decisions

01 Calculated, not self-assessed, confidence

Instead of asking the LLM to score its own certainty, a deterministic algorithm weighs agreement between specialists, quality of evidence and temporal consistency. The result is auditable round by round.

02 Critic agents as a clinical "red team"

Each specialty has one agent that proposes and another whose only goal is to find flaws in the hypotheses, which reduces confirmation bias.

03 Scheduled reports without one cron per client

Each subscription plan defines its frequency. On activation, a deferred message is scheduled in Service Bus; the worker processes it, runs the agents in parallel and schedules the next run.

04 Multi-tenant security

Each client’s FHIR credentials are resolved from Key Vault at runtime and never stored. Services authenticate with Managed Identity.

05 Reliable SignalR from Python

The orchestrator acts as just another hub client: connection on a dedicated thread, custom ping/pong, automatic reconnection and re-subscription to groups and events after every drop.

06 No duplicate answers when scaling out

With several instances, the same event can arrive twice. A Redis lock with expiry prevents calling the model twice (the expensive part) and duplicating messages.

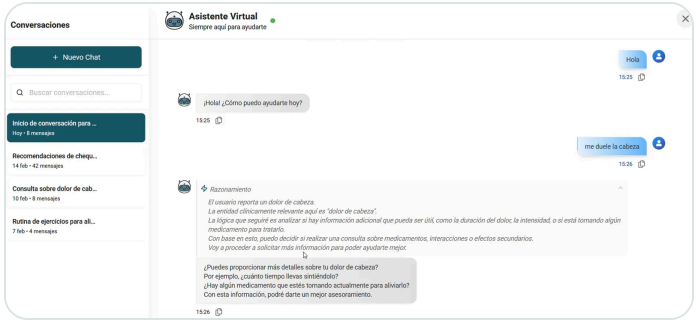
07 Memory without a latency penalty

Conversations are indexed in AI Search with embeddings after five minutes of inactivity, not on every message. That memory then powers the personalized suggestions.

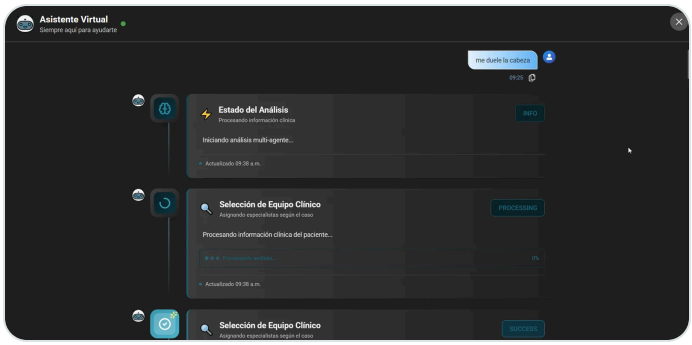
08 Refactor to Clean Architecture

The agents moved from duplicated code to a shared base class handling threads, retries and robust JSON parsing, removing close to a thousand repeated lines.

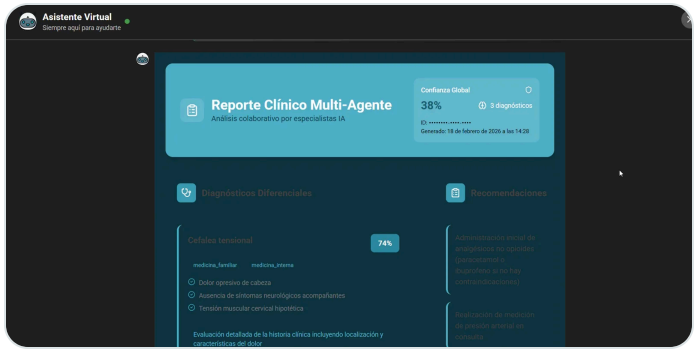
Screenshots



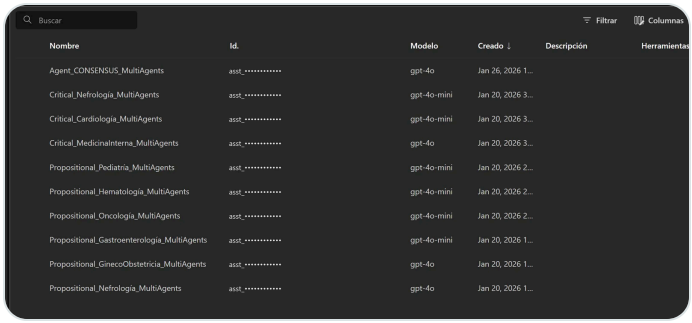
Medical chat answering (test data)



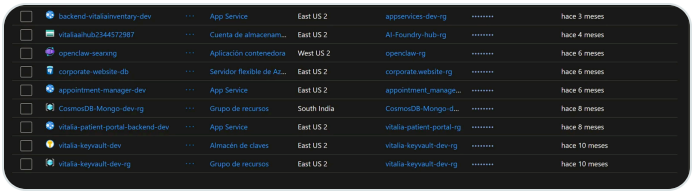
Multi-agent analysis in progress: phase, cycle and round



Final clinical report: differential diagnoses and overall confidence



Azure AI Foundry: proposing, critic and consensus agents per specialty



Platform resources on Azure: App Services, Key Vault and storage

Results

- Traceable multi-agent clinical analysis, with every round’s history available through the API.
- Automated clinical reports tailored to each subscription plan.
- Real-time feedback to the user on what the AI is doing.
- Continuous deployment to Azure with containers and pipelines.

AI AGENTS

Xuxa

AI real estate advisor · ED Bienes Raíces (Panama)

24/7 agent on WhatsApp, Telegram and web that searches properties, captures leads and answers legal questions.

WhatsApp | Telegram | Multi-agent | RAG | MCP | Azure AI Search | App Service

24/7
support across 3 channels

2
agents: advisor + legal expert

3
custom MCP servers

DEV / PROD
separate environments with rollback

My role

> End-to-end design and implementation of the agent on OpenClaw (an open-source agent gateway), extended with plugins, MCP servers, legal RAG and Azure infrastructure.

Context and problem

At ED Bienes Raíces every customer process was manual: clients waited for answers during business hours, prospects were lost without being registered, and legal questions (rentals, sales, condominium law) took up the agents’ time.

Solution

Property search

Queries the AlterEstate real estate CRM with filters for city, area, type, price and operation (sale or rent), plus projects and units.

Automatic lead capture

When it detects real interest (asking for a viewing or a price, or saying "I’m interested") it asks for contact details and registers the lead in the CRM, tagged by channel.

Lex, the legal expert

Sub-agent specialized in Panamanian real estate law. Answers with RAG over official legislation, citing the law, article and source.

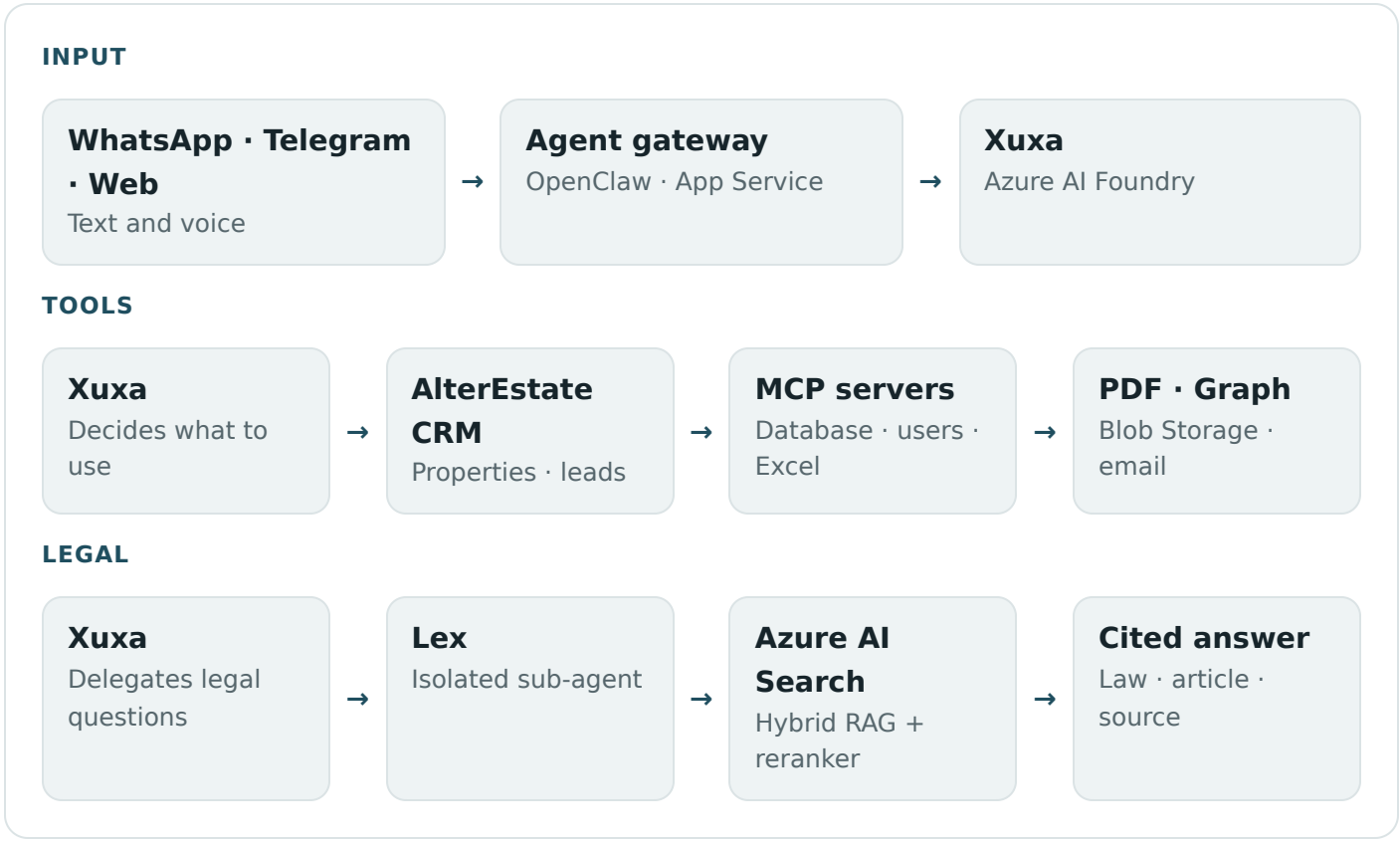
Role-based back office

Clients, agents and administrators get different tools: Excel, database, user management and, for administrators, email and calendar.

Documents and voice

Generates branded PDFs, transcribes voice notes and offers a public web demo chat.

Architecture



A containerized agent gateway receives messages from every channel and orchestrates two agents and their tools.

Journey of a message

1

Reception

Text or a voice note arrives; audio is transcribed with Azure Whisper.

2

Identification

On the first message, the user and their role are looked up in the database.

3

Reasoning

The model decides which tools to use based on intent and role.

4

Action

Searches the CRM, registers the lead, consults Lex or generates a document.

5

Response

Replies on the same channel, with attachments when needed.

6

Memory

The conversation is stored and compacted for long sessions.

Technologies

AI
Azure AI Foundry (model-router), Azure Whisper, embeddings, hybrid RAG with semantic reranker
Agent platform
OpenClaw (TypeScript, Node.js 22) with custom plugins and patches
Tools
Custom MCP servers (database, users, Excel), CRM plugin with 8 tools, PDF generator
Data
Cosmos DB for MongoDB, Azure AI Search, Blob Storage
Integrations
AlterEstate CRM (REST), Microsoft Graph, WhatsApp, Telegram

Infrastructure

App Service for containers (DEV and PROD), Container Registry, Azure Files, Container Apps, Managed Identity

Implementation: challenges and decisions

01 WhatsApp sessions being "stolen"

Loading dependencies from a network drive blocked the process for about 12 seconds; the connection missed heartbeats and WhatsApp took the session back. Local disk state was separated from shared configuration, removing the block.

02 Reliable legal RAG

Laws are chunked by article, searches use a relevance threshold and up to three reformulations before admitting there is no information. Citing the source and a legal disclaimer are mandatory.

03 Multi-agent without loops

Xuxa only delegates legal questions, and Lex works in an isolated session with a time limit, with explicit rules that prevent circular delegation.

04 Keeping a fork alive

Five upgrades of the base framework while preserving every customization. Extensions were built as plugins to reduce conflicts.

05 Defending against hallucinations

A registry of recent documents recovers files when the model "invents" paths, plus strict rules for working with Excel files and attachments.

06 Layered security

Roles resolved from the database, per-agent tool profiles, read-only MCP servers for the public chat and protection against requests to disallowed destinations.

Screenshots

Property search requested by voice note, transcribed and run against the CRM

Xuxa delegates an eviction question to Lex, the legal expert

Branded PDF generated by the agent

User and role management panel

Azure portal: DEV and PROD App Services, Foundry and services used

Admin panel: model and connected channels (Telegram and WhatsApp)

Results

- 24/7 support on WhatsApp, Telegram and web, replacing a fully manual workflow.
- Automatic lead registration in the CRM, tagged by source channel.
- Legal answers with verifiable sources for the agency’s clients.
- Controlled DEV and PROD deployments with prior backups and a documented rollback.



AI AGENTS + BACKEND

Sage

Inventory and purchasing agent for restaurants (Panama)

Staff manage inventory, purchasing and goods receiving by chatting with an agent.

MCP | Telegram | Cosmos DB | Roles and approvals | App Service | Azure Files

64
custom MCP tools

5
roles with dynamic permissions

105/105
permission tests passing

2
scheduled alerts per day

My role

> Design and implementation on OpenClaw: all business logic in custom MCP servers, gateway extensions, data model and Azure infrastructure.

Context and problem

A restaurant chain tracked inventory and purchasing in spreadsheets. Purchases had no traceable approvals, goods were received without evidence and real stock drifted from theoretical stock. Kitchen and warehouse staff work from their phones, not an ERP.

Solution

Auditable inventory

Every change is a movement (purchase, sale, production, waste, return, transfer, adjustment) recording who, when and why. Weekly stock is recalculated from the movements.

Role-based purchase approvals

Numbered requests with statuses and amount limits per role and branch. Direct purchases warn when the price has changed since the last time.

Two-step receiving

The warehouse clerk records what arrived with quality notes and a photo; another person approves or rejects each line. Only approved items move inventory.

Restock alerts

Twice a day the agent checks stock and sends a Telegram alert, distinguishing low stock from out of stock.

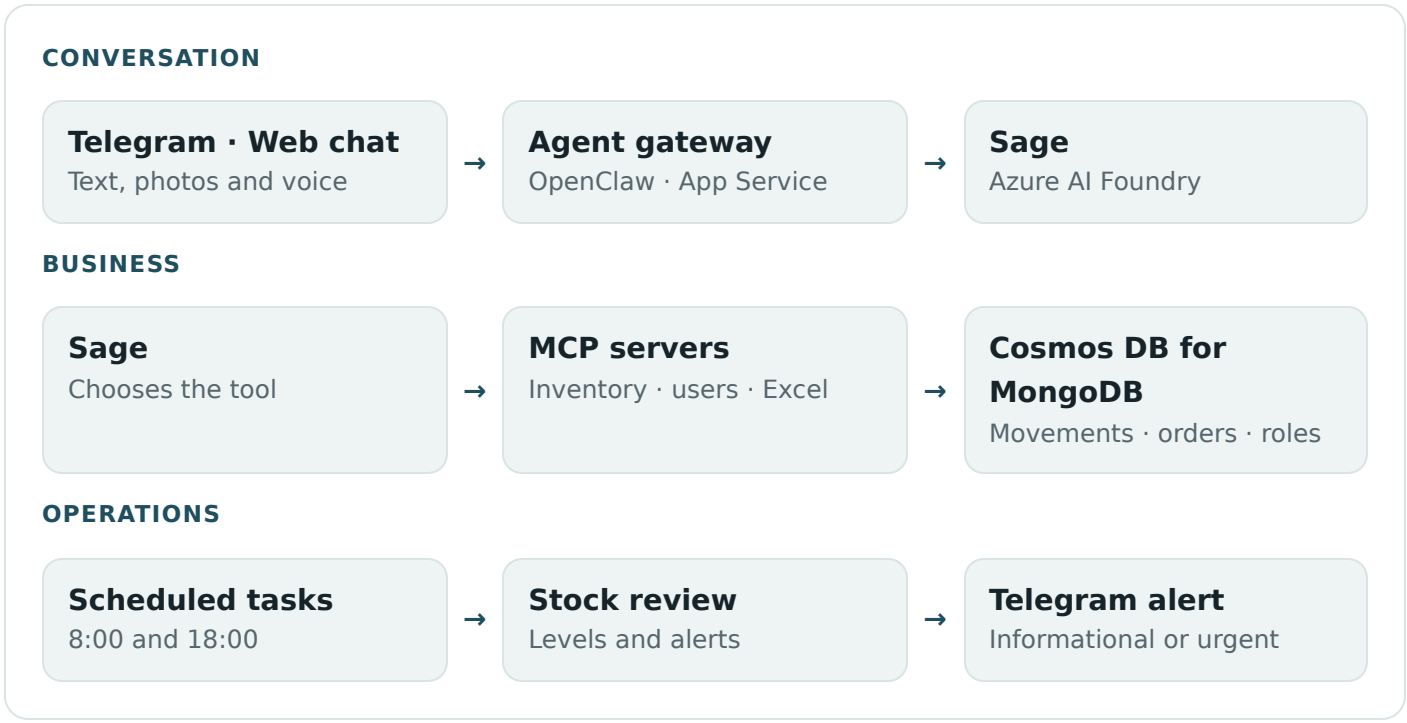
Real-world units

Converts sacks, quintals, boxes or pounds to the base unit by product and supplier. When data is missing, it asks instead of guessing.

Management and demo

Web panel for users and permissions, voice notes, Excel import and export, and a public demo chat with anonymized data.

Architecture



The agent never touches the database directly: every operation goes through MCP tools that validate role, permissions and data.

Lifecycle of a purchase request

1. **Pending**
   Created with an automatic number and branch.
2. **Approved**
   Approved by a role with a high enough amount limit.
3. **In transit**
   The supplier ships it; it can be received in parts.
4. **Receiving**
   Recorded with quality notes and photo evidence.
5. **Review**
   Another person approves or rejects each line.
6. **Received**
   Inventory is updated and the week is recalculated.

Technologies

AI
Azure AI Foundry (model-router), Azure Whisper for voice notes

Agent platform
OpenClaw (TypeScript, Node.js) with custom extensions: public chat, users API and web panel
Business logic
Custom MCP servers in Node.js: inventory and purchasing, users and permissions, Excel
Data
Cosmos DB for MongoDB (vCore) with schema validation, anonymized demo database
Infrastructure
App Service for containers, Container Registry, Azure Files (evidence and configuration)
Quality
Unit-engine tests and an end-to-end permissions test suite

Implementation: challenges and decisions

01 Not blindly trusting the model

The role is read from the database, writes require a signed confirmation bound to the exact data, and the database is pinned by configuration. This came from a real incident in which the model improvised a data structure.

02 Separation of duties

Whoever receives goods cannot approve them, and whoever proposes a product substitution cannot confirm it. This is enforced in the tools, not in the prompt.

03 Slow replies under concurrency

The bot took up to two minutes with back-to-back messages. A race condition in the framework’s session start-up was diagnosed and fixed with retries and exponential backoff.

04 Persistence in containers

Evidence photos were landing in ephemeral storage. They were moved to an Azure Files share organized by order.

05 Inventory as a ledger

Movements are the source of truth and weekly stock is a chained, recalculated view, even when a movement is recorded for a past week.

06 Safe public demo

Separate agent, anonymized database, read-only tools, rate limits and CORS. Content-filter blocks are returned as a normal refusal.

Screenshots

Purchase by chat: price-change warning and confirmation before saving

Receiving: photo evidence and line approval

Scheduled restock alert

Unit conversion: quintals, sacks, pounds and kilos

Role permissions panel

Users panel (phone numbers hidden)

Admin panel: users, roles and per-tool permissions

Results

- Purchasing and receiving with full traceability and human approval at every critical step.
- Automatic restock alerts without checking spreadsheets.
- Permissions reassignable from a web panel, taking effect in the agent immediately.
- Public demo to showcase the product without exposing real data.

BACKEND

Backend Projects

Mobility platform: geospatial search and real-time messaging

Backend services for a ride-hailing app that connects passengers and drivers.

Python · FastAPI · Azure AI Search · Service Bus · Azure SignalR · Azure Functions · Azure DevOps

2
services in production

3
Service Bus queues

4
stages in the deployment pipeline

3
delivery modes: user, group or all

My role

> Evolved the address search engine in an existing service and built the real-time messaging service end to end.

Context and problem

A mobility app needs passengers to find their destination in seconds by typing free text, and passengers and drivers to exchange events instantly even when the mobile connection is unstable.

Solution

Address search engine

Returns places and addresses ranked by relevance, popularity and proximity, with reverse geocoding, route calculation and business registration.

Queue-based processing

Searches arrive as events and are processed asynchronously with Service Bus, without blocking message intake.

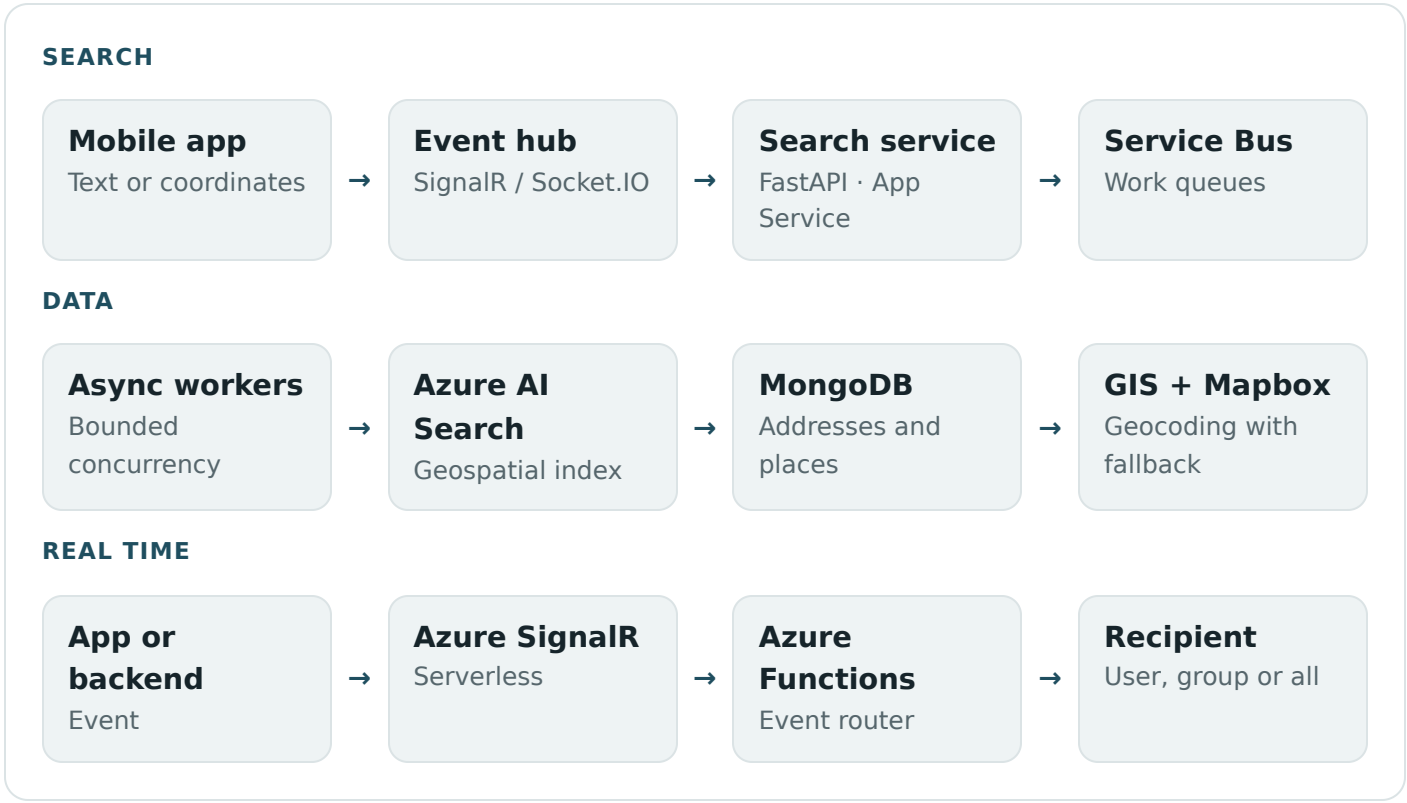
Serverless real-time messaging

Azure SignalR and Azure Functions route events to specific users, groups or everyone, without a dedicated server.

Event contracts as an artifact

The AsyncAPI specification is generated, validated and automatically published to the repositories that consume it.

Architecture



Two complementary services: event-driven geospatial search and a serverless messaging hub.

Journey of a search

1
Event
The app sends the search text through the real-time hub.

2
Validation
The message is validated and the user identified.

3
Queue
It is queued in Service Bus and the receiver is freed instantly.

4
Query
Two searches in parallel: places and addresses, with a distance filter.

5
Merge
Results are combined by relevance and popularity.

6
Response
Returned only to that user, compressed according to the app version.

Technologies

Search
Python 3.11, FastAPI, Azure AI Search (geospatial index, Spanish analyzer, scoring profiles)
Messaging
Azure Service Bus, Azure SignalR Serverless, Socket.IO
Serverless
Azure Functions in Node.js (v4 model), Azurite for local development
Data and integrations
MongoDB, Blob Storage, GIS API, Mapbox (geocoding and routing)
DevOps
Docker, Azure DevOps (4-stage pipelines), Container Registry, App Service, AsyncAPI

Observability

OpenTelemetry, Application Insights, custom queue metrics, self-healing health checks

Implementation: challenges and decisions

01 Decoupling intake and processing

The event receiver was blocked by slow searches. Service Bus was introduced with a long-lived producer and an async consumer with controlled concurrency.

02 Search relevance

Places and addresses are queried in parallel and merged with different criteria: relevance for places and popularity for addresses, with a configurable minimum threshold.

03 Declarative intents

Rules in a configuration file turn searches such as "restaurant" or "airport" into category filters, adjustable without deploying code.

04 Unstable connections

Two-layer reconnection, a custom heartbeat and a health check that tells Azure to recycle the instance if the connection does not recover.

05 Serverless messaging hub

SignalR in Serverless mode with Functions: clients invoke events handled by functions, and the event name is resolved at runtime, so new events can be added without deploying.

06 Cost-controlled observability

Telemetry sampling plus custom metrics for queue time and processing failures.

Screenshots

Address search in the app: suggestions while typing

Azure DevOps pipeline: API validation, deployment and contract publishing

AsyncAPI event contract, generated and validated in the pipeline

Azure AI Search indexes: over 160,000 addresses and places in production

Real-time Function App: SignalR and HTTP triggers

Results

- Fast, relevant address search without blocking the event channel.
- Real-time messaging that scales to zero and accepts new events without deploying.
- API contracts always in sync across backend, frontend and real time.